

Tarea 11

Nombre: David Alejandro Díaz Pineda

```
In [35]: %load_ext autoreload
%autoreload 2
from src import eliminacion_gaussiana, gauss_jacobi, gauss_seidel
import numpy as np
```

The autoreload extension is already loaded. To reload it, use:
%reload_ext autoreload

1)

Encuentre las primeras dos iteraciones del método de Jacobi para los siguientes sistemas lineales, por medio de $\mathbf{x}(0) = 0$:

$$3x_1 - x_2 + x_3 = 1$$

$$3x_1 + 6x_2 + 2x_3 = 0$$

$$3x_1 + 3x_2 + 7x_3 = 4$$

```
In [36]: A1 = np.array([[3, 1, 1], [3, 6, 2], [3, 3, 7]], dtype=float)
b1 = np.array([1], [0], [4]), dtype=float)
```

```
In [37]: soluciones = gauss_jacobi(A = A1, b = b1, x = np.zeros_like(b1), tol=1e-10, max
[01-25 16:06:51][INFO][dalz2] i= 0 x: [[0. 0. 0.]]
[01-25 16:06:51][INFO][dalz2] i= 2 x: [[ 0.14285714 -0.35714286  0.42857143]]
```

$$10x_1 - x_2 = 9$$

$$-x_1 + 10x_2 - 2x_3 = 7$$

$$-2x_2 + 10x_3 = 6$$

```
In [38]: A2 = np.array([[10, -1, 0], [-1, 10, -2], [0, -2, 10]], dtype=float)
b2 = np.array([9], [7], [6]), dtype=float)
```

```
In [39]: soluciones = gauss_jacobi(A = A2, b = b2, x = np.zeros_like(b2), tol=1e-10, max
[01-25 16:06:51][INFO][dalz2] i= 0 x: [[0. 0. 0.]]
[01-25 16:06:51][INFO][dalz2] i= 2 x: [[0.97 0.91 0.74]]
```

$$10x_1 + 5x_2 = 6$$

$$5x_1 + 10x_2 - 4x_3 = 25$$

$$-4x_2 + 8x_3 - x_4 = -11$$

$$-x_3 + 5x_4 = -11$$

```
In [40]: A3 = np.array([[10, 5, 0, 0], [5, 10, -4, 0], [0, -4, 8, -1], [0, 0, -1, 5]], dt
b3 = np.array([6], [25], [-11], [-11]), dtype=float)
```

```
In [41]: soluciones = gauss_jacobi(A = A3, b = b3, x = np.zeros_like(b3), tol=1e-10, max
```

```
[01-25 16:06:51][INFO][dalz2] i= 0 x: [[0. 0. 0. 0.]]
[01-25 16:06:51][INFO][dalz2] i= 2 x: [[-0.65  1.65 -0.4 -2.475]]
```

$$4x_1 + x_2 + x_3 + x_5 = 6$$

$$-x_1 - 3x_2 + x_3 + x_4 = 6$$

$$2x_1 + x_2 + 5x_3 - x_4 - x_5 = 6$$

$$-x_1 - x_2 - x_3 + 4x_4 = 6$$

$$2x_2 - x_3 + x_4 + 4x_5 = 6$$

```
In [42]: A4 = np.array([[4,1,1,0,0],
                       [-1,-3,1,1,0],
                       [2,1,5,-1,-1],
                       [-1,-1,-1,4,0],
                       [0, 2, -1, 1, 4]], dtype=float)
b4 = np.array([[6], [6], [6], [6], [6]], dtype=float)
```

```
In [43]: soluciones = gauss_jacobi(A = A4, b = b4, x = np.zeros_like(b4), tol=1e-10, max
[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 2 x: [[ 1.7 -1.6  1.6  1.675  2.425]]
```

2)

Repita el ejercicio 1 usando el método de Gauss-Siedel.

a.

```
In [44]: gauss_seidel(A = A1, b = b1, x = np.zeros_like(b1), tol=1e-10, max_iter=2)
[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 1 x: [[ 0.33333333 -0.16666667  0.5      ]]
[01-25 16:06:52][INFO][dalz2] i= 2 x: [[ 0.22222222 -0.27777778  0.5952381  ]]
Out[44]: array([[ 0.22222222],
                [-0.27777778],
                [ 0.5952381 ]])
```

b.

```
In [45]: gauss_seidel(A = A2, b = b2, x = np.zeros_like(b2), tol=1e-10, max_iter=2)
[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 1 x: [[0.9  0.79  0.758]]
[01-25 16:06:52][INFO][dalz2] i= 2 x: [[0.979  0.9495 0.7899]]
Out[45]: array([[0.979 ],
                [0.9495],
                [0.7899]])
```

c.

```
In [46]: gauss_seidel(A = A3, b = b3, x = np.zeros_like(b3), tol=1e-10, max_iter=2)
[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 1 x: [[ 0.6  2.2 -0.275 -2.255]]
[01-25 16:06:52][INFO][dalz2] i= 2 x: [[-0.5  2.64 -0.336875 -2.267375]]
```

```
Out[46]: array([[ -0.5      ],
               [  2.64      ],
               [-0.336875],
               [-2.267375]])
```

d.

```
In [47]: gauss_seidel(A = A4, b = b4, x = np.zeros_like(b4), tol=1e-10, max_iter=2)

[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 1 x: [[ 1.5      -2.5      1.1      1.525      2.64
375]]
[01-25 16:06:52][INFO][dalz2] i= 2 x: [[ 1.85      -1.74166667  1.64208333  1.93
760417  2.29695312]]

Out[47]: array([[ 1.85      ],
               [-1.74166667],
               [ 1.64208333],
               [ 1.93760417],
               [ 2.29695312]])
```

3)

Utilice el método de Jacobi para resolver los sistemas lineales en el ejercicio 1, con TOL = 10⁻³.

a.

```
In [48]: gauss_jacobi(A = A1, b = b1, x = np.zeros_like(b1), tol=1e-3, max_iter=2)

[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 2 x: [[ 0.14285714 -0.35714286  0.42857143]]

Out[48]: array([[ 0.14285714],
               [-0.35714286],
               [ 0.42857143]])
```

b.

```
In [49]: gauss_jacobi(A = A2, b = b2, x = np.zeros_like(b2), tol=1e-3, max_iter=2)

[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 2 x: [[0.97 0.91 0.74]]

Out[49]: array([[0.97],
               [0.91],
               [0.74]])
```

c.

```
In [50]: gauss_jacobi(A = A3, b = b3, x = np.zeros_like(b3), tol=1e-3, max_iter=2)

[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 2 x: [[-0.65  1.65 -0.4 -2.475]]

Out[50]: array([[-0.65 ],
               [ 1.65 ],
               [-0.4 ],
               [-2.475]])
```

d.

```
In [51]: gauss_jacobi(A = A4, b = b4, x = np.zeros_like(b4), tol=1e-3, max_iter=2)
[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 2 x: [[ 1.7   -1.6    1.6    1.675   2.425]]
Out[51]: array([[ 1.7   ],
                [-1.6   ],
                [ 1.6   ],
                [ 1.675 ],
                [ 2.425 ]])
```

4)

Utilice el método de Gauss-Siedel para resolver los sistemas lineales en el ejercicio 1, con $TOL = 10^{-3}$.

a.

```
In [52]: gauss_seidel(A = A1, b = b1, x = np.zeros_like(b1), tol=1e-3, max_iter=2)
[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 1 x: [[ 0.33333333 -0.16666667  0.5       ]]
[01-25 16:06:52][INFO][dalz2] i= 2 x: [[ 0.22222222 -0.27777778  0.5952381 ]]
Out[52]: array([[ 0.22222222],
                [-0.27777778],
                [ 0.5952381 ]])
```

b.

```
In [53]: gauss_seidel(A = A2, b = b2, x = np.zeros_like(b2), tol=1e-3, max_iter=2)
[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 1 x: [[0.9  0.79  0.758]]
[01-25 16:06:52][INFO][dalz2] i= 2 x: [[0.979  0.9495 0.7899]]
Out[53]: array([[0.979 ],
                [0.9495],
                [0.7899]])
```

c.

```
In [54]: gauss_seidel(A = A3, b = b3, x = np.zeros_like(b3), tol=1e-3, max_iter=2)
[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 1 x: [[ 0.6    2.2   -0.275 -2.255]]
[01-25 16:06:52][INFO][dalz2] i= 2 x: [[-0.5    2.64   -0.336875 -2.267375]]
Out[54]: array([[ -0.5    ],
                [  2.64   ],
                [-0.336875],
                [-2.267375]])
```

d.

```
In [55]: gauss_seidel(A = A4, b = b4, x = np.zeros_like(b4), tol=1e-3, max_iter=2)
```

```
[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 1 x: [[ 1.5      -2.5      1.1      1.525      2.64
375]]
[01-25 16:06:52][INFO][dalz2] i= 2 x: [[ 1.85      -1.74166667  1.64208333  1.93
760417  2.29695312]]
```

```
Out[55]: array([[ 1.85      ],
                [-1.74166667],
                [ 1.64208333],
                [ 1.93760417],
                [ 2.29695312]])
```

5)

El sistema lineal

$$2x_1 - x_2 + x_3 = -1$$

$$2x_1 + 2x_2 + 2x_3 = 4$$

$$-x_1 - x_2 + 2x_3 = -5$$

tiene la solución (1, 2, -1)t.

a. Muestre que el método de Jacobi con $x(0) = 0$ falla al proporcionar una buena aproximación después de 25 iteraciones.

```
In [56]: A = np.array([[2, -1, 1], [2, 2, 2], [-1, -1, 2]], dtype=float)
b = np.array([-1], [4], [-5]), dtype=float)
```

```
In [57]: gauss_jacobi(A = A, b = b, x = np.zeros_like(b), tol=1e-10, max_iter=25)
```

```
[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 25 x: [[-20.82787284  2.          -22.82787284]]
```

```
Out[57]: array([[-20.82787284],
                [ 2.          ],
                [-22.82787284]])
```

b. Utilice el método de Gauss-Siedel con $x(0) = 0$ para aproximar la solución para el sistema lineal dentro de 10^{-5}

```
In [58]: gauss_seidel(A = A, b = b, x = np.zeros_like(b), tol=1e-5, max_iter=25)
```

```
[01-25 16:06:52][INFO][dalz2] i= 0 x: [[0. 0. 0.]]
[01-25 16:06:52][INFO][dalz2] i= 1 x: [[-0.5 2.5 -1.5]]
[01-25 16:06:53][INFO][dalz2] i= 2 x: [[ 1.5 2. -0.75]]
[01-25 16:06:53][INFO][dalz2] i= 3 x: [[ 0.875 1.875 -1.125]]
[01-25 16:06:53][INFO][dalz2] i= 4 x: [[ 1. 2.125 -0.9375]]
[01-25 16:06:53][INFO][dalz2] i= 5 x: [[ 1.03125 1.90625 -1.03125]]
[01-25 16:06:53][INFO][dalz2] i= 6 x: [[ 0.96875 2.0625 -0.984375]]
[01-25 16:06:53][INFO][dalz2] i= 7 x: [[ 1.0234375 1.9609375 -1.0078125]]
[01-25 16:06:53][INFO][dalz2] i= 8 x: [[ 0.984375 2.0234375 -0.99609375]]
[01-25 16:06:53][INFO][dalz2] i= 9 x: [[ 1.00976562 1.98632812 -1.00195312]]
[01-25 16:06:53][INFO][dalz2] i= 10 x: [[ 0.99414062 2.0078125 -0.99902344]]
[01-25 16:06:53][INFO][dalz2] i= 11 x: [[ 1.00341797 1.99560547 -1.00048828]]
[01-25 16:06:53][INFO][dalz2] i= 12 x: [[ 0.99804688 2.00244141 -0.99975586]]
[01-25 16:06:53][INFO][dalz2] i= 13 x: [[ 1.00109863 1.99865723 -1.00012207]]
[01-25 16:06:53][INFO][dalz2] i= 14 x: [[ 0.99938965 2.00073242 -0.99993896]]
[01-25 16:06:53][INFO][dalz2] i= 15 x: [[ 1.00033569 1.99960327 -1.00003052]]
[01-25 16:06:53][INFO][dalz2] i= 16 x: [[ 0.99981689 2.00021362 -0.99998474]]
[01-25 16:06:53][INFO][dalz2] i= 17 x: [[ 1.00009918 1.99988556 -1.00000763]]
[01-25 16:06:53][INFO][dalz2] i= 18 x: [[ 0.99994659 2.00006104 -0.99999619]]
[01-25 16:06:53][INFO][dalz2] i= 19 x: [[ 1.00002861 1.99996758 -1.00000191]]
[01-25 16:06:53][INFO][dalz2] i= 20 x: [[ 0.99998474 2.00001717 -0.99999905]]
[01-25 16:06:53][INFO][dalz2] i= 21 x: [[ 1.00000811 1.99999094 -1.00000048]]
[01-25 16:06:53][INFO][dalz2] i= 22 x: [[ 0.99999571 2.00000477 -0.99999976]]
```

```
Out[58]: array([[ 1.00000226],
                [ 1.9999975 ],
                [-1.00000012]])
```

6)

El sistema lineal

$$x_1 - x_3 = 0.2$$

$$-\frac{1}{2}x_1 + x_2 - \frac{1}{4}x_3 = -1.425$$

$$x_1 - \frac{1}{2}x_2 + x_3 = 2$$

tiene la solución (0.9, -0.8, 0.7)t.

```
In [59]: A = np.array([[1, 0, -1], [-1/2, 1, -1/4], [1, -1/2, 1]], dtype=float)
          b = np.array([0.2, [-1.425], [2]], dtype=float)
```

a. ¿La matriz de coeficientes tiene diagonal estrictamente dominante?

A =

$$\begin{pmatrix} 1 & 0 & -1 \\ -\frac{1}{2} & 1 & -\frac{1}{4} \\ 1 & -\frac{1}{2} & 1 \end{pmatrix}$$

La matriz no es estrictamente dominante, porque no cumple la condición en todas las filas (falla en la fila 1 y la fila 3).

b. Utilice el método iterativo de Gauss-Seidel para aproximar la solución para el sistema lineal con una tolerancia de 10^{-22} y un máximo de 300 iteraciones.

```
In [60]: gauss_seidel(A = A, b = b, x = np.zeros_like(b), tol=1e-22, max_iter=300)
```

[01-25 16:06:53][INFO][dalz2] i= 0 x: [[0. 0. 0.]]
[01-25 16:06:53][INFO][dalz2] i= 1 x: [[0.2 -1.325 1.1375]]
[01-25 16:06:53][INFO][dalz2] i= 2 x: [[1.3375 -0.471875 0.4265625]]
[01-25 16:06:53][INFO][dalz2] i= 3 x: [[0.6265625 -1.00507812 0.87089844]]
[01-25 16:06:53][INFO][dalz2] i= 4 x: [[1.07089844 -0.67182617 0.59318848]]
[01-25 16:06:53][INFO][dalz2] i= 5 x: [[0.79318848 -0.88010864 0.7667572]]
[01-25 16:06:53][INFO][dalz2] i= 6 x: [[0.9667572 -0.7499321 0.65827675]]
[01-25 16:06:53][INFO][dalz2] i= 7 x: [[0.85827675 -0.83129244 0.72607703]]
[01-25 16:06:53][INFO][dalz2] i= 8 x: [[0.92607703 -0.78044223 0.68370185]]
[01-25 16:06:53][INFO][dalz2] i= 9 x: [[0.88370185 -0.81222361 0.71018634]]
[01-25 16:06:53][INFO][dalz2] i= 10 x: [[0.91018634 -0.79236024 0.69363354]]
[01-25 16:06:53][INFO][dalz2] i= 11 x: [[0.89363354 -0.80477485 0.70397904]]
[01-25 16:06:53][INFO][dalz2] i= 12 x: [[0.90397904 -0.79701572 0.6975131]]
[01-25 16:06:53][INFO][dalz2] i= 13 x: [[0.8975131 -0.80186517 0.70155431]]
[01-25 16:06:53][INFO][dalz2] i= 14 x: [[0.90155431 -0.79883427 0.69902855]]
[01-25 16:06:53][INFO][dalz2] i= 15 x: [[0.89902855 -0.80072858 0.70060715]]
[01-25 16:06:53][INFO][dalz2] i= 16 x: [[0.90060715 -0.79954464 0.69962053]]
[01-25 16:06:53][INFO][dalz2] i= 17 x: [[0.89962053 -0.8002846 0.70023717]]
[01-25 16:06:53][INFO][dalz2] i= 18 x: [[0.90023717 -0.79982212 0.69985177]]
[01-25 16:06:53][INFO][dalz2] i= 19 x: [[0.89985177 -0.80011117 0.70009264]]
[01-25 16:06:53][INFO][dalz2] i= 20 x: [[0.90009264 -0.79993052 0.6999421]]
[01-25 16:06:53][INFO][dalz2] i= 21 x: [[0.8999421 -0.80004343 0.70003619]]
[01-25 16:06:53][INFO][dalz2] i= 22 x: [[0.90003619 -0.79997286 0.69997738]]
[01-25 16:06:53][INFO][dalz2] i= 23 x: [[0.89997738 -0.80001696 0.70001414]]
[01-25 16:06:53][INFO][dalz2] i= 24 x: [[0.90001414 -0.7999894 0.69999116]]
[01-25 16:06:53][INFO][dalz2] i= 25 x: [[0.89999116 -0.80000663 0.70000552]]
[01-25 16:06:53][INFO][dalz2] i= 26 x: [[0.90000552 -0.79999586 0.69999655]]
[01-25 16:06:53][INFO][dalz2] i= 27 x: [[0.89999655 -0.80000259 0.70000216]]
[01-25 16:06:53][INFO][dalz2] i= 28 x: [[0.90000216 -0.79999838 0.69999865]]
[01-25 16:06:53][INFO][dalz2] i= 29 x: [[0.89999865 -0.80000101 0.70000084]]
[01-25 16:06:53][INFO][dalz2] i= 30 x: [[0.90000084 -0.79999937 0.69999947]]
[01-25 16:06:53][INFO][dalz2] i= 31 x: [[0.89999947 -0.80000039 0.70000033]]
[01-25 16:06:53][INFO][dalz2] i= 32 x: [[0.90000033 -0.79999975 0.69999979]]
[01-25 16:06:53][INFO][dalz2] i= 33 x: [[0.89999979 -0.80000015 0.70000013]]
[01-25 16:06:53][INFO][dalz2] i= 34 x: [[0.90000013 -0.7999999 0.69999992]]
[01-25 16:06:53][INFO][dalz2] i= 35 x: [[0.89999992 -0.80000006 0.70000005]]
[01-25 16:06:53][INFO][dalz2] i= 36 x: [[0.90000005 -0.79999996 0.69999997]]
[01-25 16:06:53][INFO][dalz2] i= 37 x: [[0.89999997 -0.80000002 0.70000002]]
[01-25 16:06:53][INFO][dalz2] i= 38 x: [[0.90000002 -0.79999999 0.69999999]]
[01-25 16:06:53][INFO][dalz2] i= 39 x: [[0.89999999 -0.80000001 0.70000001]]
[01-25 16:06:53][INFO][dalz2] i= 40 x: [[0.90000001 -0.79999999 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 41 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 42 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 43 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 44 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 45 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 46 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 47 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 48 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 49 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 50 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 51 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 52 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 53 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 54 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 55 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 56 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 57 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 58 x: [[0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 59 x: [[0.9 -0.8 0.7]]

[illegible]

[illegible]

[illegible]

[illegible]

```
[01-25 16:06:53][INFO][dalz2] i= 299 x: [[ 0.9 -0.8 0.7]]
[01-25 16:06:53][INFO][dalz2] i= 300 x: [[ 0.9 -0.8 0.7]]
```

```
Out[60]: array([[ 0.9],
               [-0.8],
               [ 0.7]])
```

c. ¿Qué pasa en la parte b) cuando el sistema cambia por el siguiente?

```
In [61]: B = np.array([[1, 0, -2], [-1/2, 1, -1/4], [1, -1/2, 1]], dtype=float)
```

```
In [62]: gauss_seidel(A = B, b = b, x = np.zeros_like(b), tol=1e-10, max_iter=25)
```

```
[01-25 16:06:53][INFO][dalz2] i= 0 x: [[0. 0. 0.]]
[01-25 16:06:53][INFO][dalz2] i= 1 x: [[ 0.2 -1.325 1.1375]]
[01-25 16:06:53][INFO][dalz2] i= 2 x: [[ 2.475 0.096875 -0.4265625]]
[01-25 16:06:53][INFO][dalz2] i= 3 x: [[-0.653125 -1.85820313 1.72402344]]
[01-25 16:06:53][INFO][dalz2] i= 4 x: [[ 3.64804688 0.8300293 -1.23303223]]
[01-25 16:06:53][INFO][dalz2] i= 5 x: [[-2.26606445 -2.86629028 2.83291931]]
[01-25 16:06:53][INFO][dalz2] i= 6 x: [[ 5.86583862 2.21614914 -2.75776405]]
[01-25 16:06:53][INFO][dalz2] i= 7 x: [[-5.31552811 -4.77220507 4.92942557]]
[01-25 16:06:53][INFO][dalz2] i= 8 x: [[10.05885115 4.83678197 -5.64046016]]
[01-25 16:06:53][INFO][dalz2] i= 9 x: [[-11.08092033 -8.3755752 8.89313272]]
[01-25 16:06:53][INFO][dalz2] i= 10 x: [[ 17.98626545 9.79141591 -11.0905575 ]]
[01-25 16:06:53][INFO][dalz2] i= 11 x: [[-21.98111499 -15.18819687 16.38701656]]
[01-25 16:06:53][INFO][dalz2] i= 12 x: [[ 32.97403311 19.1587707 -21.39464777]]
[01-25 16:06:53][INFO][dalz2] i= 13 x: [[-42.58929553 -28.06830971 30.55514068]]
[01-25 16:06:53][INFO][dalz2] i= 14 x: [[ 61.31028136 36.86892585 -40.87581843]]
[01-25 16:06:53][INFO][dalz2] i= 15 x: [[-81.55163687 -52.41977304 57.34175035]]
[01-25 16:06:53][INFO][dalz2] i= 16 x: [[114.88350069 70.35218793 -77.70740673]]
[01-25 16:06:53][INFO][dalz2] i= 17 x: [[-155.21481345 -98.45925841 107.9851842
5]]
[01-25 16:06:53][INFO][dalz2] i= 18 x: [[ 216.1703685 133.65648031 -147.3421283
4]]
[01-25 16:06:53][INFO][dalz2] i= 19 x: [[-294.48425668 -185.50266043 203.7329264
7]]
[01-25 16:06:53][INFO][dalz2] i= 20 x: [[ 407.66585294 253.34115809 -278.9952739
]]
[01-25 16:06:53][INFO][dalz2] i= 21 x: [[-557.79054779 -350.06909237 384.7560016
1]]
[01-25 16:06:53][INFO][dalz2] i= 22 x: [[ 769.71200321 479.62000201 -527.9020022
1]]
[01-25 16:06:53][INFO][dalz2] i= 23 x: [[-1055.60400442 -661.20250276 727.0027
5304]]
[01-25 16:06:53][INFO][dalz2] i= 24 x: [[1454.20550607 907.4284413 -998.4912854
3]]
[01-25 16:06:53][INFO][dalz2] i= 25 x: [[-1996.78257085 -1249.43910678 1374.0630
1746]]
```

```
Out[62]: array([[-1996.78257085],
               [-1249.43910678],
               [ 1374.06301746]])
```

El sistema diverge y cambia totalmente la respuesta

7)

Repita el ejercicio 11 usando el método de Jacobi.

b.

```
In [63]: gauss_jacobi(A = A, b = b, x = np.zeros_like(b), tol=1e-22, max_iter=300)
```

```
[01-25 16:06:53][INFO][dalz2] i= 0 x: [[0. 0. 0.]]
```

```
[01-25 16:06:53][INFO][dalz2] i= 300 x: [[ 0.90025541 -0.80004033  0.70012933]]
```

```
Out[63]: array([[ 0.90025541],
                [-0.80004033],
                [ 0.70012933]])
```

c.

```
In [64]: gauss_jacobi(A=B, b=b, x = np.zeros_like(b), tol=1e-22, max_iter=300)
```

```
[01-25 16:06:53][INFO][dalz2] i= 0 x: [[0. 0. 0.]]
```

```
[01-25 16:06:53][INFO][dalz2] i= 300 x: [[ 1.34478913e+43 -7.28028158e+42  1.5665
0340e+43]]
```

```
Out[64]: array([[ 1.34478913e+43],
                [-7.28028158e+42],
                [ 1.56650340e+43]])
```

8)

Un cable coaxial está formado por un conductor interno de 0.1 pulgadas cuadradas y un conductor externo de 0.5 pulgadas cuadradas. El potencial en un punto en la sección transversal del cable se describe mediante la ecuación de Laplace. Suponga que el conductor interno se mantiene en 0 volts y el conductor externo se mantiene en 110 volts. Aproximar el potencial entre los dos conductores requiere resolver el siguiente sistema lineal.

```
In [65]: A = np.array([
    [ 4, -1, 0, 0, -1, 0, 0, 0, 0, 0, 0, 0],
    [-1, 4, -1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [ 0, -1, 4, -1, 0, 0, 0, 0, 0, 0, 0, 0],
    [ 0, 0, -1, 4, 0, -1, 0, 0, 0, 0, 0, 0],
    [-1, 0, 0, 0, 4, -1, 0, 0, 0, 0, 0, 0],
    [ 0, 0, 0, -1, 0, 4, 0, -1, 0, 0, 0, 0],
    [ 0, 0, 0, 0, -1, 0, 4, 0, -1, 0, 0, 0],
    [ 0, 0, 0, 0, 0, -1, 0, 4, 0, 0, 0, -1],
    [ 0, 0, 0, 0, 0, 0, -1, 0, 4, -1, 0, 0],
    [ 0, 0, 0, 0, 0, 0, 0, 0, -1, 4, -1, 0],
    [ 0, 0, 0, 0, 0, 0, 0, 0, 0, -1, 4, -1],
    [ 0, 0, 0, 0, 0, -1, 0, 0, 0, 0, -1, 4]
], dtype=float)
```

a. ¿La matriz es estrictamente diagonalmente dominante?

La matriz es estrictamente diagonalmente dominante (por filas).

b. Resuelva el sistema lineal usando el método de Jacobi con $x(0) = 0$ y $TOL = 10^{-2}$.

```
In [66]: b = np.array([[220], [110], [110], [220], [110], [110], [110], [110], [220], [110], [110], [220]])
```

```
In [67]: gauss_jacobi(A=A, b=b, x = np.zeros_like(b), tol=1e-2, max_iter=30)

[01-25 16:06:53][INFO][dalz2] i= 0 x: [[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]

Out[67]: array([[87.99776206],
                [65.99776204],
                [65.99776204],
                [87.99776206],
                [65.99776204],
                [65.99776204],
                [65.99776204],
                [65.99776204],
                [87.99776206],
                [65.99776204],
                [65.99776204],
                [87.99776206]])
```

c. Repita la parte b) mediante el método de Gauss-Siedel.

```
In [68]: gauss_seidel(A=A, b=b, x = np.zeros_like(b), tol=1e-2, max_iter=30)

[01-25 16:06:53][INFO][dalz2] i= 0 x: [[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
[01-25 16:06:53][INFO][dalz2] i= 1 x: [[55.          41.25          37.8125          64.45
3125          41.25          43.61328125
37.8125          38.40332031  64.453125  43.61328125  38.40332031  75.50415039]]
[01-25 16:06:53][INFO][dalz2] i= 2 x: [[75.625          55.859375  57.578125  80.29
785156  57.30957031  57.17529297
57.94067383  60.66986084  80.38848877  57.19795227  60.67552567  84.46270466]]
[01-25 16:06:53][INFO][dalz2] i= 3 x: [[83.29223633  62.71759033  63.25386047  85.10
728836  62.61688232  63.9442873
63.25134277  64.60174799  85.11232376  63.94696236  64.60241675  87.13667601]]
[01-25 16:06:53][INFO][dalz2] i= 4 x: [[86.33361816  64.89686966  65.00103951  87.23
63317  65.06947637  65.45951992
65.04545003  65.64904898  87.2481031  65.46262996  65.64982649  87.7773366 ]]
[01-25 16:06:53][INFO][dalz2] i= 5 x: [[87.49158651  65.6231565  65.71487205  87.79
359799  65.73777661  65.86066174
65.74646993  65.90949959  87.80227497  65.86302537  65.91009049  87.94268806]]
[01-25 16:06:53][INFO][dalz2] i= 6 x: [[87.84023328  65.88877633  65.92059358  87.94
531383  65.92522376  65.96370335
65.93187468  65.97659785  87.94872501  65.96470388  65.97684798  87.98513783]]
[01-25 16:06:53][INFO][dalz2] i= 7 x: [[87.95350002  65.9685234  65.97845931  87.98
554067  65.97930084  65.99053463
65.98200646  65.99391812  87.98667759  65.99088139  65.99400481  87.99613486]]
[01-25 16:06:53][INFO][dalz2] i= 8 x: [[87.98695606  65.99135384  65.99422363  87.99
618956  65.99437267  65.99752692
65.99526256  65.99841544  87.99653599  65.9976352  65.99844251  87.99899236]]
[01-25 16:06:53][INFO][dalz2] i= 9 x: [[87.99643163  65.99766381  65.99846334  87.99
899757  65.99848964  65.99935325
65.99875641  65.9995864  87.9990979  65.9993851  65.99959437  87.9997369 ]]
```

```
Out[68]: array([[87.99903836],  
                [65.99937543],  
                [65.99959325],  
                [87.99973663],  
                [65.9995979 ],  
                [65.99983076],  
                [65.99967395],  
                [65.99989192],  
                [87.99976476],  
                [65.99983978],  
                [65.99989417],  
                [87.99993123]])
```